

Akadályok egy lámának

Egy láma az Andok egyik fennsíkján akar keresztül utazni. Rendelkezik a fennsík térképével, amely egy $N \times M$ cellából álló négyzetrács. A térkép sorai felülről lefelé haladva 0-tól $N - 1$ -ig, az oszlopok pedig balról jobbra haladva 0-tól $M - 1$ -ig vannak számozva. A térkép i -edik sorában és j -edik oszlopában ($0 \leq i < N, 0 \leq j < M$) található celláját az (i, j) jelöli.

A láma tanulmányozta a fennsík éghajlatát, és felfedezte, hogy a térkép minden egyes során belül az összes cellában azonos a **hőmérséklet**, és a térkép minden egyes oszlopán belül az összes cellában azonos a **páratartalom**. A láma megadott két, egész számokból álló tömböt: az N elemű T tömböt és az M elemű H tömböt. $T[i]$ az i -edik ($0 \leq i < N$) sorban lévő cellák hőmérsékletét, $H[j]$ pedig a j -edik ($0 \leq j < M$) oszlopban lévő cellák páratartalmát jelöli.

A láma tanulmányozta a fennsík növényvilágát is, és észrevette, hogy bármely (i, j) cella akkor és csak akkor **mentes a növényzettől**, ha a hőmérséklete nagyobb, mint a páratartalma, azaz $T[i] > H[j]$.

A láma a fennsíkon csak **érvényes útvonalakat** követve tud közlekedni. Egy érvényes útvonal olyan különböző cellák sorozata, amely megfelel a következő feltételeknek:

- Az útvonal bármely két egymást követő cellájának van egy közös oldala.
- Az útvonal minden cellája mentes a növényzettől.

A feladatod Q kérdésre való válaszadás. Minden kérdéshez négy egész számot kapsz: L, R, S , és D . Meg kell határozni, hogy létezik-e olyan érvényes útvonal, amelyre:

- Az útvonal a $(0, S)$ cellából indul és a $(0, D)$ cellában végződik.
- Az útvonal minden cellája a L -edik és R -edik oszlopok között van (beleértve magukat az L -edik és R -edik oszlopokat is).

Garantált, hogy a $(0, S)$ és a $(0, D)$ cella mentes a növényzettől.

Megvalósítás

Az első megvalósítandó eljárás a következő:

```
void initialize(std::vector<int> T, std::vector<int> H)
```

- T : egy N hosszúságú tömb a soronkénti hőmérsékletekkel.

- H : egy M hosszúságú tömb az oszloponkénti páratartalmakkal.
- Ez az eljárás minden egyes tesztesetben pontosan egyszer lesz meghívva, minden `can_reach` hívás előtt.

A második megvalósítandó függvény a következő:

```
bool can_reach(int L, int R, int S, int D)
```

- L, R, S, D : egy kérdést leíró egész számok.
- Ez a függvény Q -szor lesz meghívva egy tesztesetben.

Ennek a függvénynek akkor és csak akkor kell `true` (igaz) értéket visszaadnia, ha létezik érvényes útvonal a $(0, S)$ cellából a $(0, D)$ cellába, úgy, hogy az útvonal minden cellája az L -edik és R -edik oszlopok között van (a széleket is beleértve).

Korlátok

- $1 \leq N, M, Q \leq 200\,000$
- $0 \leq T[i] \leq 10^9$ minden i -re, ahol $0 \leq i < N$.
- $0 \leq H[j] \leq 10^9$ minden j -re, ahol $0 \leq j < M$.
- $0 \leq L \leq R < M$
- $L \leq S \leq R$
- $L \leq D \leq R$
- $(0, S)$ és $(0, D)$ cellák mentesek a növényzettől.

Részfeladatok

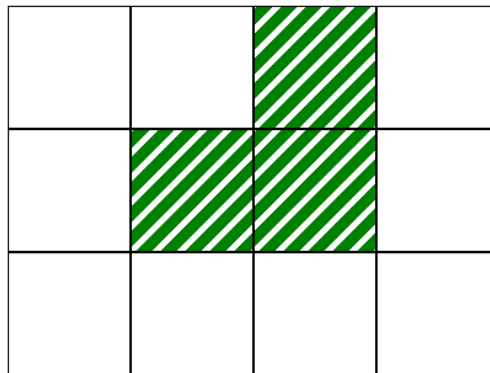
Részfeladat	Pontszám	További megszorítások
1	10	$L = 0, R = M - 1$ minden kérdésben. $N = 1$.
2	14	$L = 0, R = M - 1$ minden kérdésben. $T[i - 1] \leq T[i]$ minden i -re, ahol $1 \leq i < N$.
3	13	$L = 0, R = M - 1$ minden kérdésben. $N = 3$ és $T = [2, 1, 3]$.
4	21	$L = 0, R = M - 1$ minden kérdésben. $Q \leq 10$.
5	25	$L = 0, R = M - 1$ minden kérdésben.
6	17	Nincsenek további megszorítások.

Példa

Tekintsük a következő hívást:

```
initialize([2, 1, 3], [0, 1, 2, 0])
```

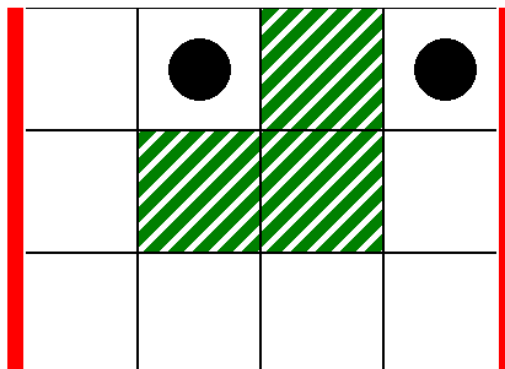
Ez megfelel a következő ábrán látható térképnek, ahol fehérrel jelöltük a növényzettől mentes cellákat:



Az első kérdésként tekintsük a következő hívást:

```
can_reach(0, 3, 1, 3)
```

Ez megfelel a következő ábrán látható helyzetnek, ahol a vastag függőleges vonalak az $L = 0$ és $R = 3$ közötti oszloptartományt jelzik, a két fekete korong pedig a kezdő és a végső cellát:



Ebben az esetben a láma a $(0, 1)$ cellából a $(0, 3)$ cellába a következő érvényes útvonalon juthat el:

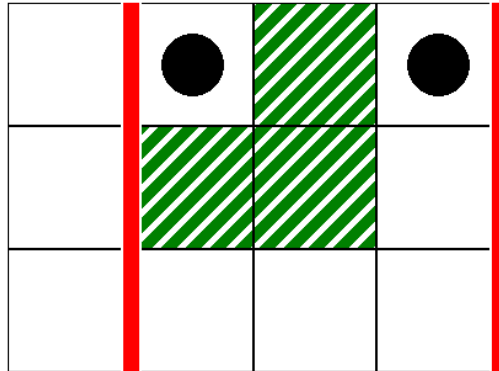
$(0, 1), (0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)$

Ezért ennek a hívásnak a `true` (igaz) értéket kell visszaadnia.

A második kérdésként tekintsük a következő hívást:

```
can_reach(1, 3, 1, 3)
```

Ez megfelel a következő ábrán látható helyzetnek:



Ebben az esetben nincs olyan érvényes útvonal a $(0,1)$ cellából a $(0,3)$ cellába, hogy az útvonal minden cellája az 1. és a 3. oszlop között legyen. Ezért ennek a hívásnak a `false` (hamis) értéket kell visszaadnia.

Mintaértékelő

Bemeneti formátum:

```
N M
T[0] T[1] ... T[N-1]
H[0] H[1] ... H[M-1]
Q
L[0] R[0] S[0] D[0]
L[1] R[1] S[1] D[1]
...
L[Q-1] R[Q-1] S[Q-1] D[Q-1]
```

Itt az $L[k]$, $R[k]$, $S[k]$ és a $D[k]$ ($0 \leq k < Q$) adja meg a `can_reach` egyes hívásainak paramétereit.

Kimeneti formátum:

```
A[0]
A[1]
...
A[Q-1]
```

Itt az $A[k]$ ($0 \leq k < Q$) értéke 1, ha `can_reach(L[k], R[k], S[k], D[k])` hívás visszatérési értéke `true` (igaz), egyébként pedig 0.